

Q.1 Which of the following is true about iterating through a set?

- A. Sets are unordered, so iteration may not follow insertion order**
 - B. You cannot iterate over a set
 - C. Sets support list slicing for iteration
 - D. Sets are indexed, and you can use the index to iterate
-

Q.2 How can multiple elements be added to a set at once?

- A. extend()
 - B. add()
 - C. update()**
 - D. append()
-

Q.3 What happens when a duplicate element is added to a set?

- A. The duplicate is ignored**
 - B. The set throws an error
 - C. The duplicate is added at the end
 - D. The set grows in size
-

Q.4 Which of the following is true about variable-length unpacking in Python?

- A. The * operator can be used with strings and tuples, but not lists
 - B. You cannot use * for unpacking a tuple
 - C. You can only use * with lists
 - D. The * operator captures all remaining elements during unpacking**
-

Q.5 How can you repeat the elements of a tuple (1, 2) three times?

- A. tuple(1, 2).repeat(3)
 - B. (1, 2).repeat(3)
 - C. (1, 2) * 3**
 - D. (1, 2) + 3
-

Q.6 How would you unpack a list l = [1, 2, 3] into individual variables?

- A. a = l[0], b = l[1]
 - B. a, *b = l
 - C. a, b = l**
 - D. a, b, c = l
-

Q.7 What does Python's consistency refer to in handling data structures?

- A. Lists and strings are treated as mutable
 - B. Similar operations are available for different data structures**
 - C. Python only supports immutable data structures
 - D. Lists and strings are immutable
-

Q.8 In which scenario "set" is most useful?

- A. When you need to access elements by index
 - B. When you need to preserve the order of elements
 - C. When you need to modify elements in place
 - D. When you need to store unique elements**
-

Q.9 Which of the following is a property of sets in Python?

- A. Sets can contain only numbers
 - B. Sets allow duplicate elements
 - C. Sets are ordered
 - D. Sets do not allow duplicate elements**
-

Q.10 How can you find the index of element 30 in a tuple t = (10, 20, 30, 40)?

- A. t.search(30)
 - B. index(t, 30)
 - C. find(30, t)
 - D. t.index(30)**
-